

BAX 423 — Homework 1 · Part 1 — ML Design

Course: Big Data (BAX 423)

Due: April 10, 2026

1. Song recommendation: frontier (closed) model vs open-source

Why you might pick a frontier / proprietary model

- **Quality and multimodal fit:** Top-tier proprietary models are often stronger at nuanced understanding of lyrics, mood, and context, which can improve relevance when cold-start data is thin.
- **Managed safety and compliance:** Vendors may offer SLAs, content policies, audit logs, and faster iteration on policy changes—useful if recommendations must avoid certain artists or regions.
- **Less operational ML burden:** No need to host large weights, manage quantization pipelines, or maintain forks—faster to ship if your team is small.

Why you might not

- **Cost and lock-in:** Per-token or per-query pricing scales with usage; switching models later can change embeddings and require full re-indexing.
- **Privacy and data residency:** Sending user listening history to a third-party API may conflict with policy unless you have strong contracts and minimization.
- **Reproducibility and research:** Open weights let you fine-tune in-house, audit behavior, and avoid black-box changes from the vendor.

2. Spotify-scale features from listening history; streaming at serve time

Feature (examples)	Training	Serving (daily “For you”)	Serving (track-to-track)
Per-user play counts / recency-weighted counts per artist, genre, mood bucket	Batch from logs	Mostly batch (nightly/hourly aggregates)	Needs near-real-time session features (last N plays in-session)

Feature (examples)	Training	Serving (daily “For you”)	Serving (track-to-track)
Sequence embeddings (e.g., last 500 track IDs → transformer / GRU embedding)	Batch jobs	Refreshed periodically + cached user vector	Streaming partial updates as user listens
Co-occurrence / item-item similarity (matrix factorization, ALS)	Batch	Precomputed; lookup at request	Hybrid: precomputed + small online adjustment for session
Demographic / region popularity	Batch	Batch	Mostly batch; can add light real-time locale context
Skip rate, completion rate per (user, track)	Batch	Batch aggregates; for live radio, streaming counters matter	

Not every feature must be streaming at serve time: playlist-style recommendations can rely on features materialized hourly or daily. Track-to-track “what goes with what I’m playing now” needs low-latency **session** and **context** features (current track, device, time of day).

3. “For you” playlist (daily/weekly) vs per-track recommendations

Batch / playlist-style

- Latency budgets are relaxed (compute overnight).
- You can use heavier models, larger candidate pools, and more expensive re-ranking because you precompute and cache a playlist.
- Exploration can be scheduled (e.g., inject novelty once per day).

Per-track / in-session

- **Stricter latency** (often tens to low hundreds of ms).
- Must emphasize **fresh session signals** (current song, short horizon of recent skips).
- **Consistency**: rapid flips between unrelated genres may hurt UX—may need smoothing or “momentum” in the policy.
- **Throughput** is higher (many more inference calls per active listener).

System requirements shift from **offline batch capacity and data pipelines** toward **online feature stores, low-latency retrieval, and robust fallbacks** when real-time signals are missing.

4. Minimizing drift when popularity and subgenres shift suddenly

- **Monitor inputs and outputs:** Track distribution of track attributes, impression/CTR distributions, and slice metrics (new artists, emerging tags). Alert on large PSI/KS shifts.
- **Freshen labels and feedback loops:** Weight recent interactions higher; use **time-decayed** training or **rolling windows** for collaborative signals.
- **Cold-start paths:** Separate policies for new artists (content-based audio/text features, similarity to clusters) so spikes don't collapse recommendations to a few megahits only.
- **Exploration:** Multi-armed bandits or epsilon-greedy layers so the system keeps probing new content instead of purely exploiting stale co-occurrence.
- **Periodic retraining and canary evaluation:** Ship candidate models on shadow traffic or small cohorts before full rollout.
- **Embedding refresh:** Rebuild or partially update item embeddings when catalog composition changes; avoid static artist vectors forever.

Together, these reduce **concept drift** (what “good” means) and **covariate drift** (what users and catalogs look like) without overreacting to single-day noise.